

Day 2 — Arrays, Stacks & Queues

Faculty: Vipul Tembulwar

Arrays & ArrayList internals

Stack (array)

Queue, Circular queue

Deque — insert at first/last

Singly linked list (start)

Today we build the first real data structures with our own hands: arrays, stacks, queues, circular queues and deques. Then we start the linked list. For each one, we also look inside the Java class that does the same job.

Structure	Rule	Java class you already use
Array / dynamic array	index → element	<code>int[]</code> , <code>ArrayList</code>
Stack	LIFO: last in, first out	<code>ArrayDeque</code> (<code>push</code> / <code>pop</code>)
Queue / circular queue	FIFO: first in, first out	<code>ArrayDeque</code> , <code>ArrayBlockingQueue</code>
Deque	both ends	<code>ArrayDeque</code> , <code>LinkedList</code>
Singly linked list	node → next node	chains inside <code>HashMap</code>

1. Arrays — the foundation

DEFINITION Array

A fixed-size collection of elements of the same type, stored in **one contiguous block** of memory and accessed by an **index** starting at 0.

Because the elements sit next to each other and all have the same size, the address of any element is one calculation away:

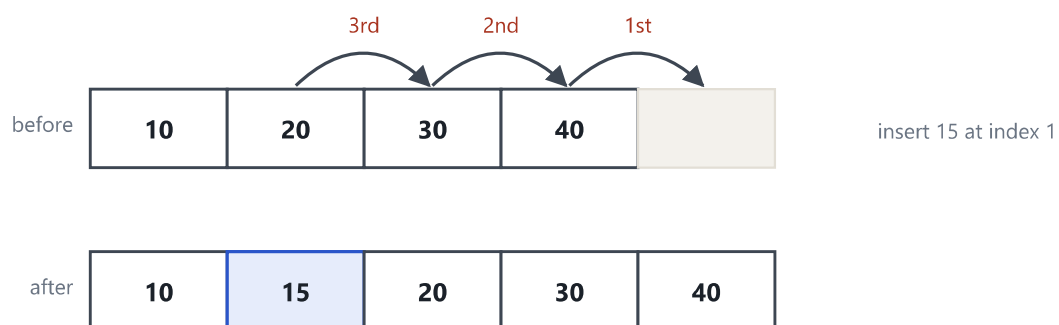
address	1000	1004	1008	1012	1016	1020
marks	12	45	7	33	90	18
	[0]	[1]	[2]	[3]	[4]	[5]

$$\text{address of marks}[3] = 1000 + 3 \times 4 = 1012$$

one multiplication and one addition, whatever the size of the array $\rightarrow O(1)$

int is 4 bytes, so element i lives at $\text{base} + i \times 4$. The JVM does this calculation for you (and you never see the real addresses).

Operation on an array of size n	Cost	Why
read / write <code>a[i]</code>	$O(1)$	address calculation
search for a value (unsorted)	$O(n)$	may need to look at every element
insert / delete at the end	$O(1)$	nothing to move (if there is space)
insert / delete at index k	$O(n)$	every element after k must shift
grow the array	$O(n)$	the size is fixed: make a bigger one and copy



Shift from the END first, otherwise you overwrite values.

Inserting at index k moves $(\text{size} - k)$ elements.

Insert in the middle = make room by shifting right. Delete = close the gap by shifting left.

JAVA `ArrayBasics.java`

```
static void insertAt(int[] a, int size, int index, int value) {
    for (int i = size; i > index; i--) { // walk from the end towards index
        a[i] = a[i - 1];                // shift one step right
    }
    a[index] = value;
}
```

```
}  
a[index] = value;  
}
```

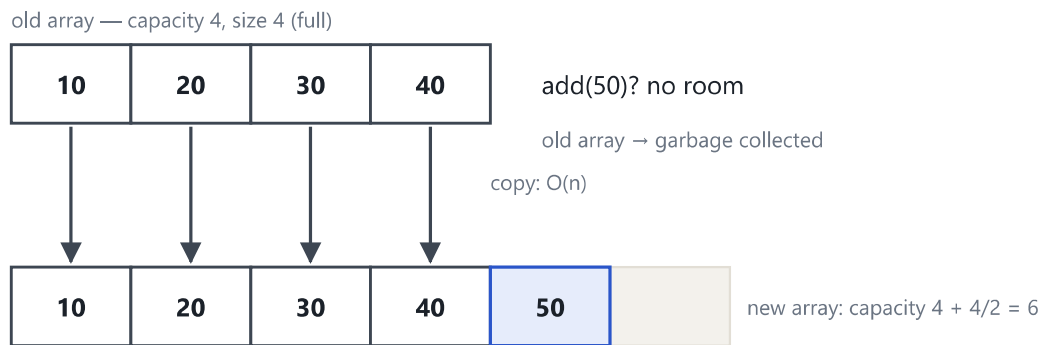
INSIDE JAVA.UTIL What Java adds to plain arrays

- An array is an **object**. `a.length` is fixed once it is created.
- New arrays are filled with default values: `0`, `false` or `null`.
- Every access is **bounds-checked**: `a[8]` on a length-8 array throws `ArrayIndexOutOfBoundsException` instead of silently corrupting memory, as C would.
- `int[][]` is an **array of arrays**, so rows can have different lengths (a jagged array).
- `System.arraycopy` and `Arrays.copyOf` are the fast built-in ways to shift and grow. `ArrayList` uses them internally.

QUICK CHECK Why does an array index start at 0?[Show answer](#)

2. The dynamic array — building our own ArrayList

An array cannot grow. A **dynamic array** hides that limit: it keeps a bigger backing array than needed and tracks how many slots are really in use (`size`). When the backing array is full, it allocates a bigger one and copies everything across.



Growth happens only at sizes 4, 6, 9, 13, 19, 28, ... so almost every `add()` is a plain $O(1)$ store.

Grow by a factor ($\times 1.5$ here, the same as `java.util.ArrayList`), never by a fixed amount

JAVA `DynamicArray.java`

```
public void add(T item) {
    if (size == data.length) grow();    // rare
    data[size++] = item;                // usual case: O(1)
}

private void grow() {
    int oldCap = data.length;
    int newCap = Math.max(oldCap + (oldCap >> 1), oldCap + 1);    // old + old/2
    data = Arrays.copyOf(data, newCap);                            // O(n) copy
}
```

Is `add` $O(1)$ or $O(n)$? One unlucky `add` costs $O(n)$. But if we count the total copying work for n adds, it stays proportional to n . Output of `DynamicArray.java`:

n adds	total elements copied	copies per add
1,000	2,131	2.13
1,00,000	2,76,515	2.77
10,00,000	20,99,747	2.10

A small constant per add means **amortised $O(1)$** .

WATCH OUT Why not grow by a fixed amount, such as +10?

Then a copy happens every 10 adds, and the copies total about $n^2/20$ elements, which is $O(n^2)$. Growing by a **factor** is what makes the average cost constant.

INSIDE JAVA.UTIL Inside `java.util.ArrayList`

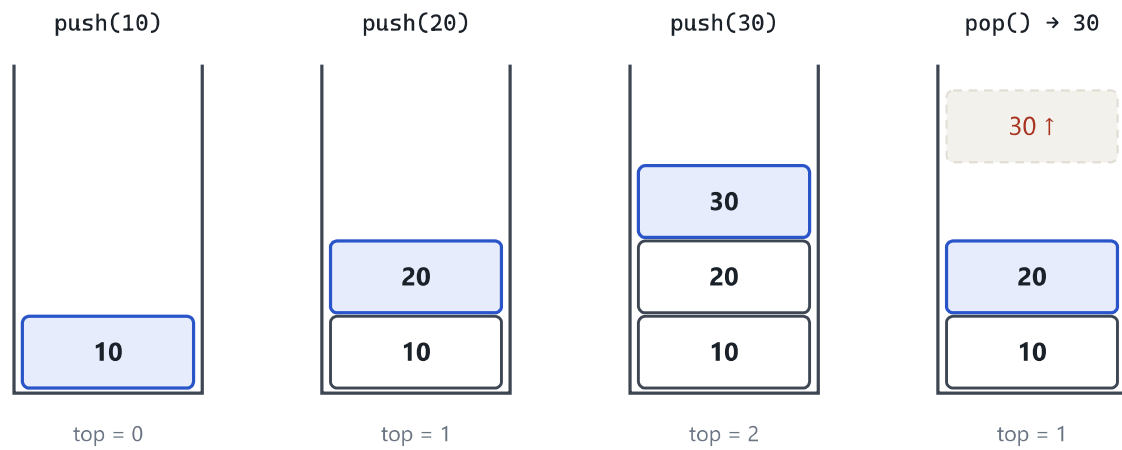
- Fields: `Object[] elementData` and `int size`. That is exactly our `data` and `size`.
- `new ArrayList<>()` starts with a shared **empty** array. The first `add` allocates capacity **10**.
- When full: `newCapacity = oldCapacity + (oldCapacity >> 1)`, which is **×1.5**, then `Arrays.copyOf`.
- `remove(i)` uses `System.arraycopy` to shift left, then sets the last slot to `null` so the GC can collect it.
- If you know the size in advance, write `new ArrayList<>(100_000)` or call `ensureCapacity(n)` to skip all the regrowth.
- It keeps a `modCount` field. Changing the list inside a for-each loop throws `ConcurrentModificationException`, a deliberate fail-fast check.

3. Stack — LIFO

DEFINITION Stack

A linear ADT where insertion (**push**) and deletion (**pop**) happen at the **same end**, called the **top**. Last In, First Out.

Real-life examples: a pile of plates, the **Undo** history of an editor, the browser's **Back** button, and the JVM's **call stack** (Day 4).



Only the top is ever touched. That is why every operation is $O(1)$.

Stack using an array (syllabus lab)

We keep an array and an index `top`, where `-1` means empty. We implement the `StackADT` contract from Day 1:

JAVA `ArrayStack.java`

```
public class ArrayStack<T> {
    private final T[] data;
    private int top = -1;           // -1 means empty

    @SuppressWarnings("unchecked")
    public ArrayStack(int capacity) {
        data = (T[]) new Object[capacity]; // "new T[capacity]" is not allowed
    }

    public void push(T item) {
        if (isFull()) throw new IllegalStateException("Stack overflow");
        data[++top] = item;           // move top up, then store
    }

    public T pop() {
        if (isEmpty()) throw new NoSuchElementException("Stack underflow");
        T item = data[top];
        data[top--] = null;           // no loitering: let the GC reclaim
    }
}
```

```

        return item;
    }

    public T peek()          { if (isEmpty()) throw new NoSuchElementException()
    public boolean isEmpty() { return top == -1; }
    public boolean isFull()  { return top == data.length - 1; }
    public int size()        { return top + 1; }
}

```

Operation	Cost	Error case
push	O(1)	overflow when full (fixed-size array)
pop	O(1)	underflow when empty
peek, isEmpty, size	O(1)	—

NOTE Two Java details worth knowing

1. **Generic arrays:** Java erases `T` at runtime, so `new T[n]` does not compile. The standard workaround is `(T[]) new Object[n]`. `ArrayList` simply stores an `Object[]`.
2. **Loitering:** if `pop` only did `top--`, the array would still hold a reference to the popped object, and the garbage collector could never free it. Setting the slot to `null` fixes that. The JDK does the same thing.

Stack applications

1. Balanced brackets. Push every opening bracket. On a closing bracket, the top of the stack **must** be its partner.

JAVA `BalancedBrackets.java`

```

static boolean isBalanced(String s) {
    Deque<Character> stack = new ArrayDeque<>();
    for (char ch : s.toCharArray()) {
        if (ch == '(' || ch == '[' || ch == '{') {
            stack.push(ch);

```

```

    } else if (ch == ')' || ch == ']' || ch == '}') {
        if (stack.isEmpty()) return false;           // closing with nothing
        if (!matches(stack.pop(), ch)) return false; // wrong partner
    }
}
return stack.isEmpty();                             // anything left open
}

```

Input	Result	What catches it
{[(())]}	balanced	—
([)]	not balanced	[is on top when) arrives
((a + b)	not balanced	the stack is not empty at the end
a + b)	not balanced	a closing bracket arrives while the stack is empty

2. Expression evaluation. Humans write **infix** (`3 + 4 * 2`). Computers prefer **postfix** (`3 4 2 * +`), which needs no brackets and no precedence rules while evaluating. A stack does both jobs:

- **Infix → postfix:** operands go straight to the output. Operators wait on a stack until an operator of lower precedence (or a `)` pushes them out.
- **Evaluate postfix:** push numbers. On an operator, pop two, apply it, and push the result.

Evaluating `3 4 2 * +`:

Token	Action	Stack after (top at right)
3	push	3
4	push	3 4
2	push	3 4 2
*	pop 2 and 4, push 4×2	3 8
+	pop 8 and 3, push $3 + 8$	11

Result: **11**. Output of `PostfixEvaluation.java`:

TEXT

$3 + 4 * 2$	postfix: $3\ 4\ 2\ * +$	= 11
$(3 + 4) * 2$	postfix: $3\ 4 + 2 *$	= 14
$10 - 4 - 3$	postfix: $10\ 4 - 3 -$	= 3
$2 ^ 3 ^ 2$	postfix: $2\ 3\ 2 ^ ^$	= 512

QUICK
CHECK

When evaluating $10\ 4 -$, we pop 4 first and then 10. Which is the left operand?

[Show answer](#)

INSIDE JAVA.UTIL

Which stack should you use in real Java code?

`Deque<Integer> stack = new ArrayDeque<>();` with `push`, `pop` and `peek`.

`ArrayDeque` is a resizable **circular array** (Section 6), with $O(1)$ operations and no synchronization overhead. Avoid `java.util.Stack`: it extends `Vector`, as we saw on Day 1.

4. Queue — FIFO

DEFINITION Queue

A linear ADT where insertion (**enqueue**) happens at the **rear** and deletion (**dequeue**) happens at the **front**. First In, First Out.

Real-life examples: a ticket counter, a printer's job list, the CPU scheduler's ready queue, requests waiting for a web server, and breadth-first search in graphs (Day 9).

Operation	Meaning	Cost
<code>enqueue(x)</code>	add at the rear	$O(1)$
<code>dequeue()</code>	remove from the front	$O(1)$
<code>peek()</code>	look at the front	$O(1)$

Operation	Meaning	Cost
<code>isEmpty()</code> , <code>isFull()</code> , <code>size()</code>	status	$O(1)$

Linear queue on an array — and its problem

We keep two indexes: **front** (first element) and **rear** (last element). Enqueue does **rear++** . Dequeue does **front++** . Both indexes only ever move **forward**...



rear == capacity - 1, so the queue reports "full"... while 3 slots are empty!

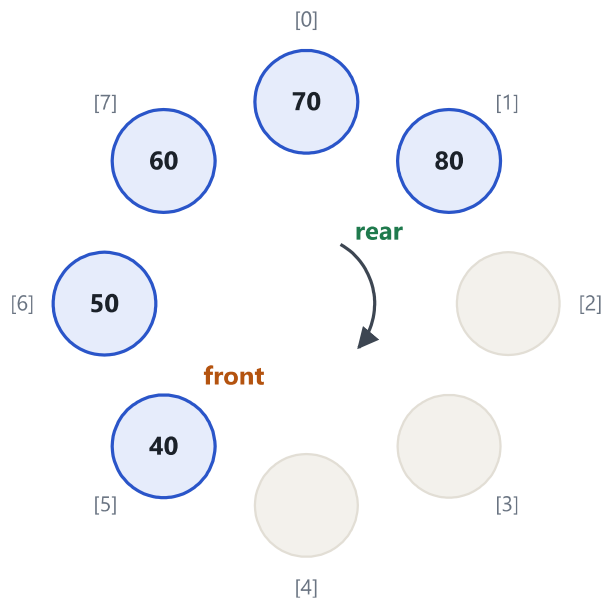
After enqueue 10, 20, 30, 40, 50 and dequeue $\times 3$. Shifting everything left would fix it, but that costs $O(n)$ per dequeue.

5. Circular queue (syllabus lab)

Idea: treat the array as a **ring**. When an index reaches the end, it wraps around to 0. The `%` (modulo) operator does the wrapping:

JAVA

```
next = (i + 1) % capacity;    // 7 → 0 when capacity is 8
```



capacity = 8, front = 5, size = 5
 rear = (front + size - 1) % 8 = 1
 next free slot = (rear + 1) % 8 = 2
 after index 7 comes index 0

empty : size == 0
 full : size == capacity

The same array, drawn as a ring. The queue runs 40 → 50 → 60 → 70 → 80, wrapping from index 7 to index 0.

How do we tell "empty" from "full"? With only `front` and `rear`, both states can end up with the two indexes in the same position. There are two standard fixes:

Approach	Empty when	Full when	Note
keep a count (<code>size</code>)	<code>size == 0</code>	<code>size == capacity</code>	simplest; used by <code>ArrayBlockingQueue</code>
keep one slot always empty	<code>front == rear</code>	<code>(rear + 1) % cap == front</code>	wastes one slot, needs no counter

We use the count version:

JAVA `CircularQueue.java`

```
private final Object[] data;
private int front = 0;    // index of the first element
private int size = 0;     // number of elements

public void enqueue(T item) {
    if (isFull()) throw new IllegalStateException("Queue overflow");
    int rear = (front + size) % data.length;    // first free slot after the last
```

```

    data[rear] = item;
    size++;
}

public T dequeue() {
    if (isEmpty()) throw new NoSuchElementException("Queue underflow");
    T item = (T) data[front];
    data[front] = null;
    front = (front + 1) % data.length;    // wrap around
    size--;
    return item;
}

```

Output of `CircularQueue.java` (F = front, R = rear):

TEXT

```

enqueue 10..50    [ 10F | 20  | 30  | 40  | 50R ] size=5
dequeue x3        [  .  |  .  |  .  | 40F | 50R ] size=2
enqueue 60 (wrap) [ 60R |  .  |  .  | 40F | 50  ] size=3
enqueue 70        [ 60  | 70R |  .  | 40F | 50  ] size=4
enqueue 80        [ 60  | 70  | 80R | 40F | 50  ] size=5 full=true

```

The freed slots are **reused**. Every operation is still $O(1)$.

INSIDE JAVA.UTIL Circular arrays inside the JDK

- `java.util.concurrent.ArrayBlockingQueue` is a fixed-capacity circular queue with fields `items[]`, `takeIndex`, `putIndex` and `count`. That is exactly our design (front, rear, size), plus thread safety.
- `java.util.ArrayDeque` is a circular array with `head` and `tail` indexes that **grows** when it fills up. It is Java's go-to class for both queues and stacks.

6. Deque — insert and delete at both ends

The syllabus lab asks for a "*queue with inserting element at different location (First, Last)*". That structure is a **deque** (double-ended queue, pronounced "deck").



Stack = addFirst + removeFirst (one end only)

Queue = addLast + removeFirst (in at one end, out at the other)

One structure that can act as both a stack and a queue

Built on a circular array, `addFirst` just moves `front` one step **back**, wrapping from 0 to the last index:

JAVA `CircularDeque.java`

```
public void addFirst(T item) {
    if (size == data.length) grow();
    front = (front - 1 + data.length) % data.length;    // step back, wrapping
    data[front] = item;
    size++;
}

public void addLast(T item) {
    if (size == data.length) grow();
    data[(front + size) % data.length] = item;
    size++;
}

public T removeLast() {
    int last = (front + size - 1) % data.length;
    T item = (T) data[last];
    data[last] = null;
    size--;
}
```

```
return item;
}
```

**QUICK
CHECK**

Why `(front - 1 + data.length) % data.length` and not just `(front - 1) % data.length`?

[Show answer](#)

When the deque is full, `grow()` copies the elements **in order** (front first) into an array twice the size, starting again at index 0. This is rare, so the operations stay amortised $O(1)$.

INSIDE JAVA.UTIL The Deque interface — 12 methods, one clear pattern

	First end: throws	First end: returns special value	Last end: throws	Last end: returns special value
insert	<code>addFirst</code>	<code>offerFirst</code>	<code>addLast</code>	<code>offerLast</code>
remove	<code>removeFirst</code>	<code>pollFirst</code>	<code>removeLast</code>	<code>pollLast</code>
examine	<code>getFirst</code>	<code>peekFirst</code>	<code>getLast</code>	<code>peekLast</code>

`push` / `pop` / `peek` work on the **first** end. The `Queue` methods `offer` / `poll` add at the **last** end and remove from the **first**. `ArrayDeque` rejects `null` elements, because `poll()` uses `null` to mean "empty".

7. Singly linked list — a first look

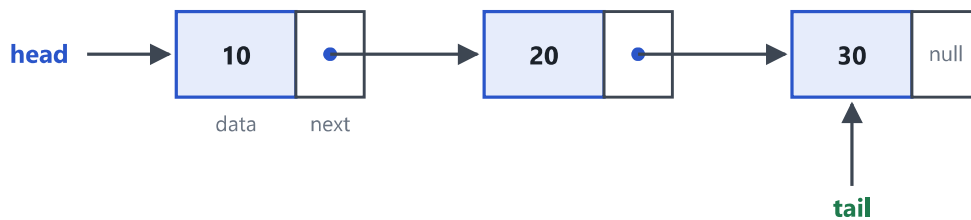
Arrays have two weaknesses: a **fixed size**, and **shifting** on every insert or delete in the middle. A linked list removes both. Each element lives in its own **node**, and every node holds a reference to the next one.

Insert n items at the **front** (`FrontInsertRace.java`):

n	<code>ArrayList.add(0, x)</code>	<code>LinkedList.addFirst(x)</code>	<code>ArrayDeque.addFirst(x)</code>
50,000	113 ms	7 ms	2 ms

n	ArrayList.add(0, x)	LinkedList.addFirst(x)	ArrayDeque.addFirst(x)
1,00,000	452 ms	1 ms	1 ms
2,00,000	1,840 ms	5 ms	4 ms

Nodes can be anywhere in memory. Only the references (arrows) connect them.



head → 10 → 20 → 30 → null. Keeping a tail reference makes addLast O(1).

JAVA SinglyLinkedList.java

```

public class SinglyLinkedList<T> implements Iterable<T> {

    private static class Node<T> {    // static: a node does not need the outer class
        T data;
        Node<T> next;
        Node(T data) { this.data = data; }
    }

    private Node<T> head, tail;
    private int size;

    public void addFirst(T x) {        // O(1)
        Node<T> n = new Node<>(x);
        n.next = head;                // 1. new node points to the old head
        head = n;                     // 2. head moves to the new node
        if (tail == null) tail = n;
        size++;
    }

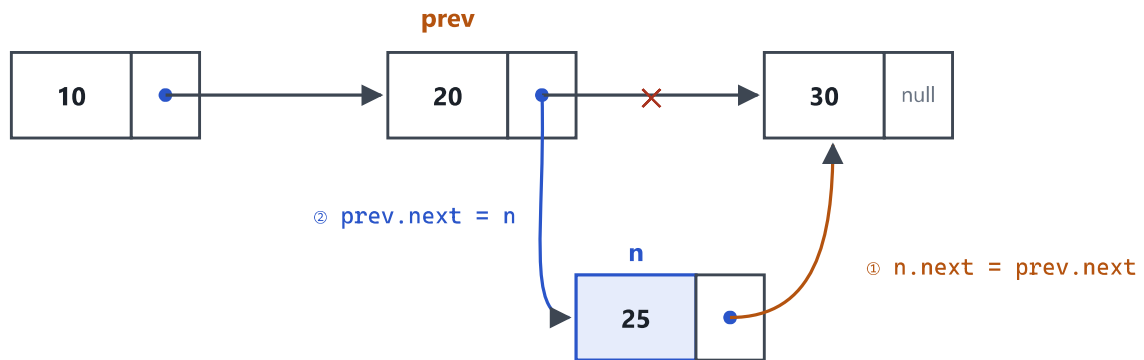
```

```

    }
}

```

Inserting in the middle — order matters



add(index, x): walk to the node before the position ($O(n)$), then relink two references ($O(1)$)

JAVA

```

public void add(int index, T x) {
    if (index == 0) { addFirst(x); return; }
    if (index == size) { addLast(x); return; }
    Node<T> prev = nodeAt(index - 1);    // O(n) walk
    Node<T> n = new Node<>(x);
    n.next = prev.next;                  // ① link the new node first
    prev.next = n;                       // ② then redirect prev
    size++;
}

```

Operations and their cost

Operation	Singly linked list (head + tail)	Array / ArrayList
<code>get(i)</code>	$O(n)$: walk from the head	$O(1)$
<code>addFirst</code>	$O(1)$	$O(n)$: shift everything

Operation	Singly linked list (head + tail)	Array / ArrayList
<code>addLast</code>	O(1) with a tail reference	amortised O(1)
<code>add(i, x)</code>	O(n) to walk, O(1) to link	O(n) to shift
<code>removeFirst</code>	O(1)	O(n)
<code>removeLast</code>	O(n) : must find the node before the tail	O(1)
search	O(n)	O(n); O(log n) if sorted (Day 6)
extra memory	one <code>next</code> reference per node	spare capacity at the end

`removeLast` is O(n) because a singly linked list **cannot go backwards**. Tomorrow's **doubly** linked list fixes that with a `prev` reference.

Reversing a list — the classic interview question

Walk the list once and turn every arrow around. We need three references: `prev`, `cur` and `next`.

JAVA

```

public void reverse() {                                // 0(n) time, 0(1) extra space
    Node<T> prev = null, cur = head;
    tail = head;
    while (cur != null) {
        Node<T> next = cur.next;                        // 1. remember the rest of the list
        cur.next = prev;                                // 2. turn this arrow around
        prev = cur;                                     // 3. move prev forward
        cur = next;                                     // 4. move cur forward
    }
    head = prev;
}

```

Step	prev	cur	List so far
start	null	10	10 → 20 → 30
1	10	20	null ← 10, and 20 → 30 still to do

Step	prev	cur	List so far
2	20	30	null ← 10 ← 20, and 30 still to do
3	30	null	null ← 10 ← 20 ← 30, so head = 30

INSIDE JAVA.UTIL **Iterable** — why for-each works on our list

Our class `implements Iterable<T>` and returns an `Iterator` that walks `cur = cur.next`. That is the same contract every `java.util` collection follows, so `for (int x : list)` works on **our** list too. You will also meet singly linked nodes inside `HashMap`: each bucket holds a chain of `Node<K,V>` objects linked by `next` (Day 8).

8. Summary

- **Array**: contiguous memory, $O(1)$ index access, $O(n)$ insert or delete in the middle, fixed size.
- **Dynamic array**: grow by a **factor**, so `add` is amortised $O(1)$. That is `ArrayList` ($\times 1.5$).
- **Stack (LIFO)**: push/pop/peek at the top, all $O(1)$. Used for brackets, expressions, undo and the call stack. In Java, use `ArrayDeque`.
- **Queue (FIFO)**: enqueue at the rear, dequeue at the front. A linear array queue wastes slots, so use a **circular** queue with `%`.
- **Circular queue**: tell empty from full with a count (`size`) or with one empty slot.
- **Deque**: both ends in $O(1)$, so one structure works as a stack or a queue. That is `ArrayDeque`.
- **Singly linked list**: nodes plus references. $O(1)$ at the head (and at the tail with a tail reference), $O(n)$ for index access and `removeLast`. **Order of relinking matters.**

9. Lab sheet — Day 2

The code files for these tasks will be shared by your faculty. Run a file directly with `java FileName.java` (Java 11+), or open it in your IDE.

LAB Task 1 — Implement a stack using an array (syllabus lab)

Type `ArrayStack<T>` yourself; don't copy it. Then write a **menu-driven** program (push / pop / peek / display / exit) that uses it. Test overflow and underflow on purpose.

LAB Task 2 — Queue with insertion at First and Last (syllabus lab)

Implement `CircularDeque<T>` with `addFirst`, `addLast`, `removeFirst`, `removeLast`, `peekFirst` and `peekLast`. Print the **raw array** plus `front` and `size` after every operation, and check that `addFirst` on index 0 wraps to the end.

LAB Task 3 — Circular queue (syllabus lab)

Implement `CircularQueue<T>` with the **count** approach. Then write a second version with the **one empty slot** approach (no `size` field). Run the same sequence of operations on both and compare the outputs.

LAB Task 4 — Stack applications

1. Run `BalancedBrackets.java` and add three test cases of your own.
2. Extend `PostfixEvaluation.java` to support `%`, and report a clear error for malformed input such as `3 +`.
3. Reverse a string using your `ArrayStack<Character>`.

LAB Task 5 — Singly linked list

Add these methods to `SinglyLinkedList<T>` and state the Big O of each: `T middle()` (the middle element in **one** pass: move one pointer by 1 and another by 2), `int count(T x)`, and `void insertSorted(T x)` for a list of `Integer` kept in ascending order.

LAB Task 6 — Measure it

Run `FrontInsertRace.java` and `DynamicArray.java`. In one sentence each, explain the numbers using Big O.

TRY IT Home challenge

1. **Queue using two stacks** — enqueue pushes onto stack A. Dequeue pops from stack B, and if B is empty, first moves everything from A into B. Why is this amortised $O(1)$?
2. **Min-stack** — a stack that also returns the minimum element in $O(1)$. Hint: use a second stack.
3. Reverse a linked list **recursively**. We will need it on Day 4.

10. Quick quiz

QUICK CHECK 1. Push 1, 2, 3, pop, push 4, pop, pop. What is left on the stack?

Show answer

QUICK CHECK 2. A circular queue has capacity 6, front = 4 and size = 3. At which index does the next enqueue go?

Show answer

QUICK CHECK 3. Why is `ArrayList.add` called "amortised $O(1)$ " and not simply " $O(1)$ "?

Show answer

QUICK CHECK 4. Which is faster for `removeLast()` : an `ArrayList` or our singly linked list?

Show answer

QUICK CHECK 5. You need a structure for a "recently viewed products" feature: add at the front, remove the oldest from the back when there are more than 10. Which Java class would you pick?

Show answer

Next class — Day 3: Linked Lists. Doubly and circular linked lists, node storage using arrays, stack and circular queue built on linked lists, and a look inside `java.util.LinkedList`.